

4.2 Arduino 万用表的设计与制作

一、实验目的

- 1、学习掌握用 Arduino UNO 设计万用表；
- 2、学习掌握 PCB 电路板的设计和制作；
- 3、学习掌握 Arduino UNO 扩展板的设计与制作；
- 4、学习掌握 LCD1602 液晶显示屏的使用。

二、实验设计方案

本项目使用 Arduino UNO 设计万用表。硬件框图如图 4.7 所示，下面对各部分模块进行说明。

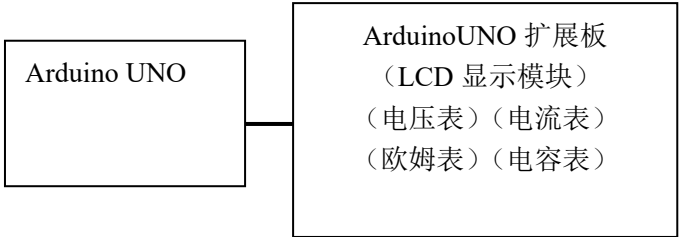


图 4.7 Arduino 万用表框图

LCD 显示模块

本次实验使用 Arduino UNO 直接驱动 1602 液晶显示字母和数字，LCD 显示模块有一个优势就是将驱动电路集成在模块中。LCD 显示模块具有标准，不同的生产商所生产的大多数模块都可以按相同方法来使用。1602 液晶在应用中非常广泛，最初的 1602 液晶使用的是 HD44780 控制器，现在各个厂家的 1602 模块基本上都是采用与之兼容的 IC，所以特性上基本都是一致的。有关 LCD1602 模块的内容，详见附录 1。

三、硬件设计

万用表由电压表、电流表、欧姆表和电容表等功能模块组成，下面就各个部分进行说明：

1、电压表

三量程可以通过 Arduino 扩展板加开关进行选择。在进行电压测量时，只有一个开关是闭合的。

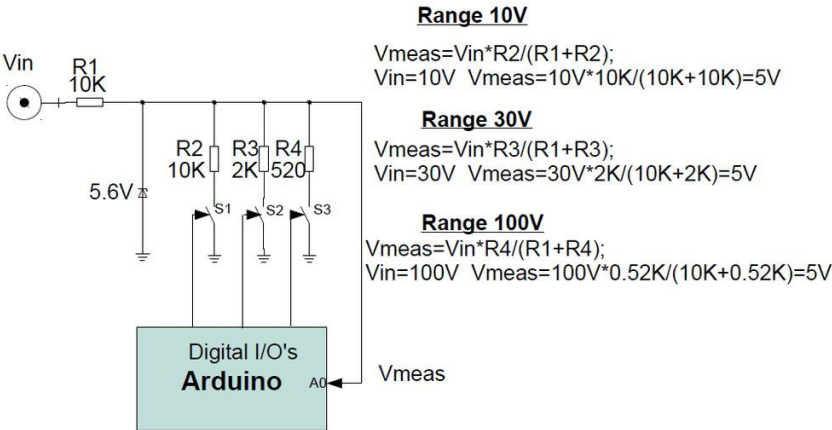


图 4.8 电压表电路简图

2、电流表

被测量电流流过 1Ω 电阻到地，其输出经放大后连接到 Arduino 的 A1 接口，放大器的增益为 10。为了进行过流保护，配置了一个 500mA 的复位 PTC。

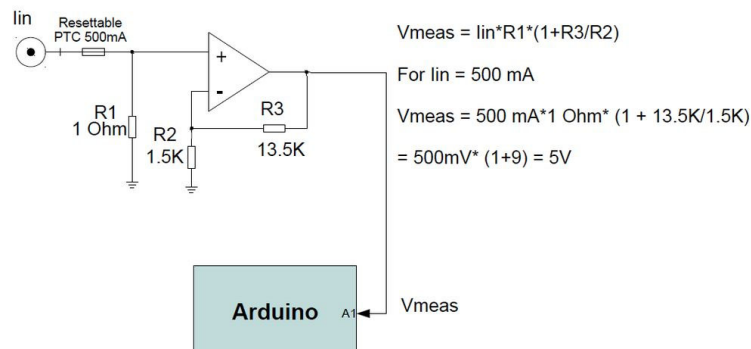


图 4.9 电流表电路简图

3、欧姆表

由齐纳二极管相对于正电压源产生基准电压，该电压被施加到由晶体管和运算放大器组成的电流转换器上，作为电压输入，晶体管的射极接可变电阻。此时齐纳二极管上的电压和该电阻上的电压相等。

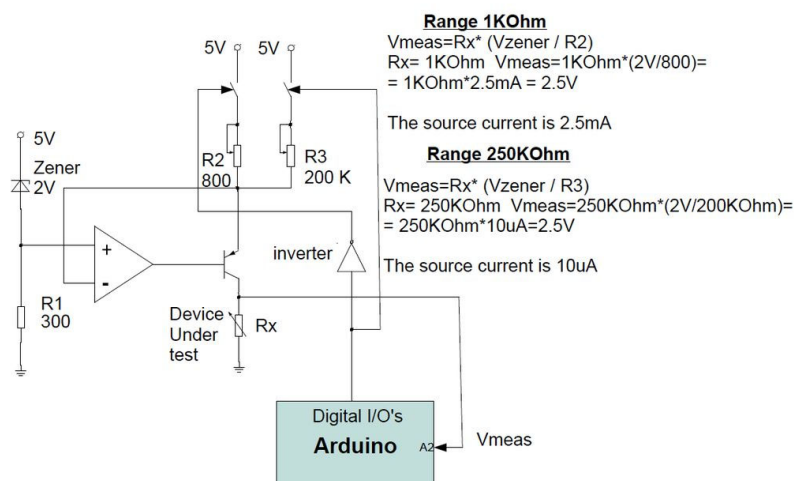


图 4.10 欧姆表的简图

“Arduino”板控制两个开关中的哪一个闭合，以这种方式决定两个电阻中的哪一个将流过电流。因此，两个可能的电流值是： $10\mu\text{A}$ 、 2.5mA 。这些电流可以非常精确地调节。这样产生的电流通过被测器件(电阻、二极管、LED 和晶体管等)，在被测器件(DUT)上电压降由“Arduino”板的模拟引脚 A2 检测。

4、电容表

电容的测量是基于对 RC 电路的时间常数的计算，由脉冲电路原理可知，电容的充电速度与 R 和 C 的大小有关，R 与 C 的乘积越大，过渡时间就越长。这个 RC 的乘积就叫做 RC 电路的时间常数 τ ，即 $\tau = R \cdot C$ 。若 R 的单位用欧姆，C 的单位用法拉，则 τ 的单位为秒。

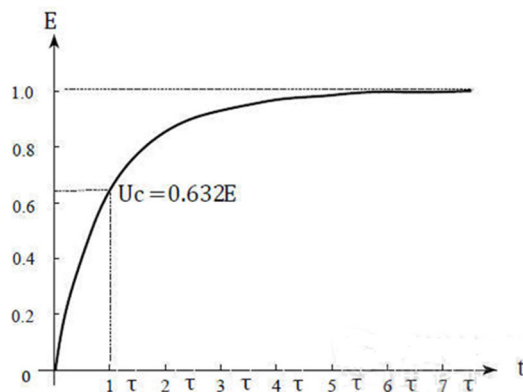


图 4.11 电容测量原理

图示曲线可以得到充电过程的一般规律： U_c 是按指数规律上升的， U_c 开始变化较快，以后逐渐减慢，并缓慢地趋近其最终值，当 $t = \tau$ 时， $U_c = 0.632E$ ；本测量仪就是利用 Arduino 测量 $U_c = 0$ 到 $0.632E$ 这段时间，用下列式子计算被测电容值：

$$C = \frac{\tau}{R} \quad (4-1)$$

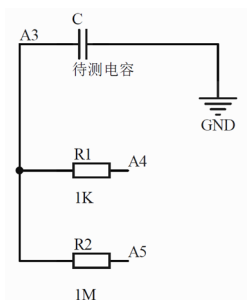


图 4.12 电容测量原理图

四、软件设计

我们给出电容测试程序，供大家参考。其它各部分程序，请大家自己设计。

程序中首先导入 LCD1602 库，硬件按 4 数据口模式连线。

```
#include <LiquidCrystal.h> //声明 1602 液晶的函数库
```

```
LiquidCrystal lcd(12,11,5,4,3,2); //4 数据口模式连线声明
```

导入库后，定义了 3 个使用的引脚的常量。引脚 A3 被用作模拟输入，测量电容两端的电压。待测电容根据电容值大小，选择通过 $1\text{ k}\Omega$ 电阻 R_1 或 $1\text{ M}\Omega$ 电阻 R_2 充电。这两个电阻通过引脚 A4 和 A5 控制。如果 A4 或 A5 引脚被设定为输入，则与之相连的电阻不参与电容的充电或放电过程。如果它被设定为输出并置低电位，电容将通过它放电。如果它设为输出和高电位，则电容将通过它充电。

```
const int stepHighCPin=A4;
```

```
const int stepLowCPin=A5;
```

```
const int sensePin=A3;
```

常量也用于定义两个电阻的值。

```
const float R1=1000.0;
```

```
const float R2=1000000.0;
```

threshold 常量是在记录用 R_1 或 R_2 给电容充电所需时间的时候，被认为是已经充分充电的模拟读数值(0 到 1023)。这就是时间常数，是 5 V 满挡读数的 63%。

```
const int threshold=645; //63%
```

本项目电容测量仪的基本原理是，测量电容器充电达到一定的水平需要的时间。这要求模拟测量要尽可能快。

下面的代码行，先使测量从每秒约 9500 个读数加快到每秒 68000 个读数。

```
//speed up ADC
const byte PS_128=(1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);
const byte PS_16=(1<<ADPS2);
void setup()
{
  ADCSRA &= ~ PS_128; // 清除分频系数 128
  ADCSRA |= PS_16; // 设置分频系数为 16
  pinMode(stepHighCPin, INPUT); //设置电阻 R1 的控制引脚为输入
  pinMode(stepLowCPin, INPUT); //设置电阻 R2 的控制引脚为输入
  lcd.begin(16,2); //初始化 LCD 显示器
  lcd.clear();
  lcd.print("Capacitance");//在顶行显示信息 Capacitance
}
```

loop 函数首先需要使用 readC 函数读取高电容值的电容。如果这个读数太低了，它快速地切换量程，灵敏度提高 1000 倍，再读取另一个读数。然后调用 display 在 LCD 上显示读数，等待半秒之后再次启动。

```
void loop()
{
  float highRange = readC(R1, stepHighCPin, stepLowCPin);
  float reading=highRange;
  if(highRange<1.0F)
  {
    float lowRange=readC(R2, stepLowCPin, stepHighCPin);
    reading=lowRange;
  }
  display(reading);
  delay(500);
}
```

该 readC 函数首先调用 discharge 放空电容器的电荷，然后测量通过指定的引脚(stepPin)需要多长时间来给电容充电。电容的微法值可以通过所用时间的微秒数除以所用电阻的欧姆值来计算。

```
float readC(float r, int stepPin, int unusedPin)
{
  discharge();
  long t0=micros();
  charge(stepPin, unusedPin);
  long t1=micros();
  float T=float(t1-t0);
  float C=T / r;
```

```

    return C;
}

```

我们希望放电要尽可能快，所以放电总是使用 $1k\Omega$ 电阻 R_1 进行。放电函数启用 R_1 ，然后等待直到电容的模拟读数达到 0。

```

void discharge()
{
    pinMode(stepLowCPin, INPUT);
    pinMode(stepHighCPin, OUTPUT);
    digitalWrite(stepHighCPin, LOW);
    while(analogRead(sensePin)>0){};
}

```

当在电容器充电测试时，被使用的电阻引脚设置为输出和 HIGH。未使用的电阻控制引脚设置为输入。charge 函数主要是等待循环，直到读数达到阈值。

```

void charge(int stepPin, int unusedPin)
{
    pinMode(unusedPin, INPUT);
    pinMode(stepPin, OUTPUT);
    digitalWrite (stepPin, HIGH);
    while(analogRead(sensePin) < threshold) {};
}

```

在 LCD 上显示内容有 3 种可能途径，由 display 函数控制。如果读数低于 $0.001(1nF)$ ，显示屏显示 Too Low。如果它大于 $0.8\mu F$ ，那么它将以微法显示读数；否则以纳法显示。

```

void display(float reading)
{
    if(reading<0.001)
    {
        clearRow();
        lcd.print("Too Low");
    }
    else if(reading>0.8)
    {
        clearRow();
        lcd.print(reading);
        lcd.print( "uF" );
    }
    else
    {
        clearRow();
        lcd.print(reading*1000.0);
        lcd.print("nF");
    }
}

```

clearRow 函数在显示新的值之前用来清空显示器的底行。

```
void clearRow()
{
    lcd.setCursor(0,1);
    lcd.print( "  ");
    lcd.setCursor(0,1);
}
```

五、实验任务与要求

1、用 Arduino Uno 板设计万用表，测量数据可以用 LCD 显示模块显示。该万用表应具备以下功能：

- 具备量程 0-30V 的电压表
- 具备 0-100mA 量程的电流表
- 具备 0-250k Ω 量程的欧姆表
- 测量大于 1nF 到几百 μ F 的电容值

2、设计的电路，完成相应器件的选择和参数计算，制作 Arduino UNO 扩展板。

3、编制与调试万用表的程序。

4、将制作的 Arduino UNO 扩展板与 Arduino UNO 板组装后，进行系统联调。

5、用一个标准的万用表，对组装的万用表进行校准。